In Ruby it would look like this:

```
Given /^the flying distance between (.*) and (.*) is (\d+) km$/ do
     |departure, destination, distance|
  ...
end
```

The test automation library will read the feature files and figure out what method it should call for each step. You can also tell the library how to extract important data out of the text and pass that data to the step definition method. But it's the step definition's job to do whatever needs to be done to perform this step.

Because the feature files and annotations consist of free text, there's always a risk that when the text in a feature file is modified, you may forget to update the text in the annotation, or vice versa. In this case, the affected scenario or scenarios will be flagged as pending once again. To help developers manage this sort of issue, many modern IDEs have plugins for BDD tools like JBehave, Cucumber, and SpecFlow, which highlight steps in scenarios that don't have matching methods.

6.1.2 Keep the step definition methods simple

A step definition method is like the conductor of an orchestra: it knows at a high level how to perform a task, and it coordinates calls to other more application-specific libraries that do the more detailed work.

Step definitions should be clean, simple, and descriptive: they should describe what you need to do to the application, or what you need to ask it. For any but the most trivial of applications, you should regroup the code that manipulates the application into a layer that's separate from the step definitions themselves. A typical architecture involves a minimum of three layers:

- *Scenarios* describe the high-level requirement:

```
Scenario: Earning standard points from an Economy flight
Given the flying distance between Sydney and Melbourne is 878 km
And I am a standard Frequent Flyer member
When I fly from Sydney to Melbourne
Then I should earn 439 points
```

- *Step definitions* interpret the scenario texts and call the test automation layer to perform the actual tasks:

```
@Given("the flying distance between $a and $b is $distance km")
public void defineTheFlyingDistanceForATrip(...) {...}
```

- *The test automation layer* interacts with the application under test:

```
inTheTestDatabase.theDistanceBetween(departure)
                 .and(destination)
                 .is(distance);
```

This architecture is illustrated in figure 6.3.